THE TURNSTILE-ORACLE CAUSAL RING

A decoupled synchronization and storage architecture, proved in Lean 4

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
CONCURRENT SYSTEMS

EMBEDDED_TYPES
2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

ABSTRACT

Concurrent queues couple synchronization to storage: a compare-and-swap loop or a per-slot sequence counter guards every slot, and the proof of the queue is the proof of that coupling. We separate the two. A *turnstile* is a manifold of lanes, each one atomic word on its own cache line. A reader enters with one fetch-or, leaves with one fetch-add, and a writer drains by reading each open lane once and waiting on the busy ones. A *ticket ring* is a power-of-two array of slots that a 64-bit stamp indexes with one mask. Sixteen flag bits of the stamp are the state machine of its slot, and every transition is one compare-and-swap that succeeds for exactly one caller, with no loop. Every slot records the stamp that wrote it, so the ring is a log of causal events as well as a queue.

The C11 implementation is seven headers. The Lean 4 formalization proves the lane word at 32 and 64 bits, the open count, the queue word, the backoff, the drain and the sleeps of the turnstile, and the stamp, the place, the window and the pushes of the ring. GenMC checks the drain and the resize of the manifold under five memory models. From these the paper derives the rest with no new mechanism: a worker's identity is the constant address it enters by, its time is the nanosecond clock the backoff reads, a rewind of k tickets is one pop at ticket $t - k$ inside the window, and an upset in a stamp is detected by the stamp that the pop echoes back or by the one claim per slot that a flipped worker and the owner of a ticket collide on. A verified memory manager, the cellblock of MMgr, is built over the turnstile and lifts the drain theorem into its own Lean proof as one whole step. We state each theorem as it is checked and name the three trusted axioms: the atomics of the target, the futex contract of the host, and the depth bound that the grantor of tickets keeps.

CONTENTS

1 THE TURNSTILE-ORACLE CAUSAL RING 1

1.1 1. Introduction 1

1.2 2. System model and the composite words 2

2.1 Two composite words 2

2.2 The ring 3

2.3 The turnstile 5

2.4 Memory ordering 5

2.5 The sleep and the futex contract 6

1.3 3. Formal verification in Lean 4 6

3.1 The lane word (`Manifold/LaneWord.lean`) 7

3.2 The open count and the manifold (`Manifold/Open.lean`) 8

3.3 The queue word (`Manifold/Queue.lean`) 9

3.4 The drain and the sleeps (`Manifold/Drain.lean`, `Manifold/Wake.lean`) 9

3.5 The backoff (`Manifold/Backoff.lean`) 9

3.6 The ticket ring (`TicketRing.lean`) 10

3.7 Model checks (`test/prove`) 11

3.8 Coarse-grained refinement 12

3.9 The claim of a slot (`Claim.lean`) 12

3.10 The flag matrix (`Flags.lean`) 13

3.11 The device arm (`Device.lean`) 14

1.4 4. Case study: the cellblock memory manager over the turnstile 15

1.5 5. Related work 16

1.6 6. Limitations and conclusion 17

BIBLIOGRAPHY 19

1

THE TURNSTILE-ORACLE CAUSAL RING

This paper describes and verifies a decoupled synchronization and storage architecture. The synchronization half is the *turnstile*, a manifold of lanes in `include/embed_turnstile.h`. The storage half is the *ticket ring*, a power-of-two array of slots in `include/embed_ticket_ring.h`. Both are C11 headers of the embedded_types library ISO C11 [1]. The formalization is the Lean 4 project under `theory/embedded_types` Lean 4 [2], and the model checks are the GenMC clients under `test/prove` GenMC [3]. Every claim below is one of three kinds, and the paper names the kind each time: a theorem that Lean checks, an execution space that GenMC exhausts under a named memory model, or an axiom that the artifact trusts from the target, the host or the caller.

1.1 1. INTRODUCTION

A bounded concurrent queue is usually one object. The Vyukov multi-producer multi-consumer queue Vyukov [4] stores a sequence counter beside each slot, and a producer claims a slot with a compare-and-swap loop over the enqueue position, then publishes the slot by a release store of the counter. The LMAX Disruptor Disruptor [5] couples a ring, a sequence barrier and a wait strategy into one framework. In both, the proof of the data structure is the proof of the coupling: a slot is correct because the counter next to it says so, and the counter is correct because every path that writes the slot also writes the counter.

We take the opposite position. The ring holds no counter and no per-slot sequence. A slot is one 64-bit stamp word and one job pointer, and every transition of the slot is one compare-and-swap on the stamp from the word with the flag bits of the transitions before it to the word with its own. The compare-and-swap is not a loop: it succeeds for exactly one caller and fails for every other, and a failure is a result, not a retry. The guard against the writer is outside the ring: the turnstile. A worker enters a turnstile before it touches the ring and leaves after. The entry is one atomic fetch-or on a lane word, the leave is one atomic fetch-add on the same word. A writer that wants to retire storage swaps a published pointer and *drains*: it reads each open lane once, and waits on each lane that a pass holds until that pass ends. The drain is the only wait in the architecture, and the only place where one thread observes the progress of another.

The decoupling buys three things.

- **A small ABI.** The ring exports a stamp, a place, a push and a pop, and two wrappers that put the push and the pop inside a pass. The turnstile exports an entry, a leave, a drain, a lane count and a backoff. There is no wait strategy to configure and no sequence barrier to compose.

- **A small proof.** The ring is proved as a function from a list of pushes to a ring state, with no concurrency in the model, because the turnstile keeps each push and each pop inside a pass. The turnstile is proved as a transition system over one lane word, one open count and one queue word. The two proofs meet at one obligation: a pass is exclusive with respect to the drain that waits for it.
- **A causal log.** Every slot records the 64-bit stamp of the push that wrote it. A pop returns that stamp beside the job. The stamp is injective in its two fields, so a consumer can tell the ticket that wrote a slot from the ticket it expected, which turns the ring into an append-only record of causal events over its window. A rewind of k events is one pop at the ticket k back, and a damaged stamp is one compare after the pop (Section 3.6).

The rest of the paper is the system model (Section 2), the Lean 4 formalization with its theorems (Section 3), the verified consumer built over the interface, the cellblock memory manager of MMgr, whose Lean proofs import the turnstile's and take its drain as one whole step (Section 4), related work (Section 5), and the trust boundary (Section 6).

1.2 2. SYSTEM MODEL AND THE COMPOSITE WORDS

2.1 Two composite words

The architecture packs its state into two 64-bit words, each one a composite of fields.

The **lane word** is the state of one turnstile lane. From bit 0 up, its fields are:

bits	field	meaning
0	busy	a pass, a contraction or a growth holds the lane
1	closed	a contraction closed the lane; it lies at or above the open count
2	sleeper	a thread sleeps on the word with the futex contract
3 up	generation	one count for each pass end, each close, each release and each takeover; rolls at the width

The C defines the fields as `EMBED_TURNSTILE_BUSY`, `EMBED_TURNSTILE_CLOSED`, `EMBED_TURNSTILE_SLEEPER` and `EMBED_TURNSTILE_GENERATION`, with the generation shift at 3. Each lane is one `embed_atomic_word`, the atomic word as wide as the machine word, aligned to 64 bytes so that no lane shares a cache line with another:

```
typedef struct
{
    EMBED_ALIGN(64) embed_atomic_word word;
} EmbedTurnstileLane;
EMBED_STATIC_ASSERT(sizeof(EmbedTurnstileLane) == 64u, "a lane takes one cache line");
```

The **stamp** is the capability that the ring indexes by. It is one atomic 64-bit word: the epoch in bits 48 to 63, a 16-bit flag field in bits 32 to 47, and the ticket in bits 0 to 31. Five flag bits are the transitions of the slot state machine, one is the parity of the job pointer, seven hold the lane index of the pop, and bit 47 is reserved and never set:

```
#define EMBED_TICKET_MASK ((embed_u64)0xffffffffu)
#define EMBED_TICKET_FLAG_MASK ((embed_u64)0xffffu << 32)
#define EMBED_TICKET_EPOCH_SHIFT 48u
#define EMBED_TICKET_GRANTED ((embed_u64)1u << 32)
#define EMBED_TICKET_PUSHED ((embed_u64)1u << 33)
#define EMBED_TICKET_POPPED ((embed_u64)1u << 34)
#define EMBED_TICKET_REPLAYED ((embed_u64)1u << 35)
#define EMBED_TICKET_RELEASED ((embed_u64)1u << 36)
#define EMBED_TICKET_PARITY ((embed_u64)1u << 37)
#define EMBED_TICKET_LANE_SHIFT 40u
#define EMBED_TICKET_RESERVED ((embed_u64)1u << 47)

EMBED_INLINE embed_u64 embed_ticket_stamp(embed_u16 epoch, embed_u32 ticket)
{
    return ((embed_u64)epoch << EMBED_TICKET_EPOCH_SHIFT) | (embed_u64)ticket;
}
```

The epoch is 16 bits because a compare succeeds only where the ticket and the epoch both match, and the ticket alone distinguishes any two live stamps. The epoch therefore only has to outlast the longest hold of a stamp, which is the bound the grantor keeps as part of Axiom A3. At a 16 µs tick it covers one second, and the grantor derives it from `embed_clock_ns` by a shift.

The two words divide the capability between them. The lane word carries the waiter flag (sleeper), the hold flag (busy) and the generation; the stamp carries the epoch, the ticket, and the state of its slot. Two more values complete it and cost nothing, because the turnstile already requires them. The identity of a worker is the constant address it enters by: `embed_turnstile_enter` takes `address`, an immutable place such as the worker's own state, and selects the start lane from it (`start_shift`). The time of a worker is `embed_clock_ns`, the nanosecond clock that the host defines for the backoff and that every worker can read at the cost of one call. A grantor that folds that clock into the epoch field gives every stamp a time order, and a consumer that reads a stamp back from a slot (Section 2.2) reads the ticket, the epoch and, through the slot, the job that a worker at a known address pushed at a known time. The detection of a damaged stamp is the same read-back, Section 3.6.

2.2 The ring

A ring is a mask and a pointer to 2^s slots that the caller owns:

```
typedef struct
{
    embed_atomic_u64 stamp;
    void *job;
} EmbedTicketRingSlot;
EMBED_STATIC_ASSERT(sizeof(EmbedTicketRingSlot) == 16u, "a slot is two words");

typedef struct
{
    EMBED_ALIGN(64) embed_u64 mask;
    EmbedTicketRingSlot *slots;
} EmbedTicketRing;
```

The place of a stamp is its ticket masked to *s* bits. Every transition of a slot is one compare-and-swap on its stamp word. The expected value is the input stamp with the flag bits of the transitions before, and the new value adds the flag bit of the transition. An input stamp with any flag bit set is invalid, and the expected value then carries the reserved bit, which no slot word has:

```
EMBED_INLINE embed_u64 embed_ticket_ring_expected(embed_u64 stamp, embed_u64 flags)
{
    const embed_u64 invalid = (embed_u64)((stamp & EMBED_TICKET_FLAG_MASK) != 0u);

    return stamp | flags | (invalid << 47u);
}

EMBED_INLINE embed_bool embed_ticket_ring_transition(EmbedTicketRingSlot *slot, embed_u64 expected,
                                                     embed_u64 value, embed_u64 *seen)
{
    embed_u64 found = expected;
    const embed_bool success = embed_atomic_u64_swap_if(&slot->stamp, &found, value);

    *seen = found;
    return success;
}

EMBED_INLINE embed_bool embed_ticket_ring_push(EmbedTicketRing *ring, embed_u64 stamp, void *job,
                                               embed_u64 *seen)
{
    EmbedTicketRingSlot *const slot = embed_ticket_ring_slot(ring, stamp);
    const embed_u64 expected = embed_ticket_ring_expected(stamp, EMBED_TICKET_GRANTED);

    slot->job = job;
    return embed_ticket_ring_transition(slot, expected, expected | EMBED_TICKET_PUSHED, seen);
}

EMBED_INLINE embed_bool embed_ticket_ring_pop(EmbedTicketRing *ring, embed_u64 stamp, unsigned int lane,
                                              void **job, embed_u64 *seen)
{
    EmbedTicketRingSlot *const slot = embed_ticket_ring_slot(ring, stamp);
    const embed_u64 expected = embed_ticket_ring_expected(stamp, EMBED_TICKET_GRANTED | EMBED_TICKET_PUSHED);
    const embed_u64 value = expected | EMBED_TICKET_POPPED | ((embed_u64)lane << EMBED_TICKET_LANE_SHIFT);
    const embed_bool success = embed_ticket_ring_transition(slot, expected, value, seen);

    *job = success ? slot->job : NULL;
    return success;
}
```

The state sequence of a slot is grant, push, pop, replay, release, one transition each: `embed_ticket_ring_grant` from a zero or released word, `embed_ticket_ring_push`, `embed_ticket_ring_pop` with the lane index of the pass, `embed_ticket_ring_replay` for a rewind, and `embed_ticket_ring_release` by the writer after its drain. The job pointer is stored before the push's compare-and-swap, which has release semantics, and read after the pop's, which has acquire semantics. The push also folds the parity of the job pointer into one flag bit of the new value, and the pop and the replay fold the pointer they read and compare it with that bit. A pop whose compare succeeds but whose job pointer fails the parity returns no job and marks the reserved bit in the value it

reports, so no caller runs a corrupted job and the fault is distinguishable from contention, since no slot word ever holds the reserved bit. The check is a six-step shift-and-xor fold, and `EMBED_TICKET_JOB_PARITY` turns it off. The shift is at most `EMBED_TICKET_RING_SHIFT_MOST`, which is 31, so the mask never reaches the flag field.

2.3 The turnstile

A set of turnstiles is `EmbedTurnstiles`: `EMBED_TURNSTILE_LANES` lanes (64 by default, and a build can set 1 to 64), an open count `open` that holds the number of open lanes less one, and a queue word `queue` that counts the workers that found every open lane busy. A set of zeros is at rest with one lane open.

An **entry** by a worker at address a reads the open count n, starts at lane $(a \gg 6) \bmod n$, and tries a fetch-or of the busy bit on each lane in turn. The fetch-or takes the lane where the old value has busy clear; where the old value has busy set, the fetch-or changed nothing and the entry spills to the next lane modulo n. Where every open lane was busy, the worker adds one to the queue word and tries to grow the manifold: a compare-and-swap on lane n from free or from closed to busy, then a compare-and-swap of `open` from n to $n + 1$. A growth that lands gives the worker lane n as its pass. A growth that fails backs off (Section 2.5) and tries again. On its way out of the entry a worker that was in the queue subtracts one from the queue word, and the entry ends with a sequentially consistent fence.

A **leave** on lane k is one fetch-add of `GENERATION - BUSY`, which clears busy and adds one to the generation in a single indivisible step. Where the old value had the sleeper bit, the leave clears that bit with a fetch-and and wakes every thread asleep on the word. Where k was the top open lane, the leave contracts the manifold while the queue word reads zero, more than one lane is open and no pass holds the top lane. A contraction claims the top lane with the same fetch-or an entry uses, swaps `open` down by one, and closes the lane with a fetch-add of `GENERATION + CLOSED`. The chain of contractions after one leave is the *cascade*.

A **drain** by a writer runs after the writer has swapped a published pointer. It issues a sequentially consistent fence, reads the open count, reads each open lane once into a snapshot, and then for each lane whose snapshot has busy set and closed clear it waits until busy reads clear or the generation differs from the snapshot. Every pass that began before the fence of the drain and still holds a lane at the fence is below the open count that the drain reads (Theorem 3.3), so the snapshot covers every such pass, and the generation makes each wait end at the end of the pass that the snapshot saw and not at a later pass on the same lane.

2.4 Memory ordering

The atomics header selects one of four arms. On the C11 arm, a load is `memory_order_acquire`, a store is `memory_order_release`, every read-modify-write is `memory_order_acq_rel`, and the fence `EMBED_ATOMIC_ORDERED()` is `atomic_thread_fence(memory_order_seq_cst)`. The GCC and clang builtin arm uses the same orders through `__atomic_*`. The MSVC arm uses the interlocked intrinsics, which are full barriers on x86 and x64, and a full barrier for the fence. The CUDA arm uses the device atomics and `__threadfence()`.

The ordering argument of the architecture is two fences. The entry of a worker ends with a fence after its fetch-or claims a lane, before the pass reads any data of the published pointer. The drain of a writer begins with a fence after the writer swapped the pointer, before the drain reads the open count and the lanes. With both fences sequentially consistent, every execution orders either the claim of the worker before the fence of the drain, in which case the drain reads the lane as busy and waits for the leave of that pass, or the fence of the drain before the claim, in which case the worker reads the new pointer. The job pointer is ordered by the stamp word itself. The push stores the job pointer and then performs its compare-and-swap, which has release semantics; the pop performs its compare-and-swap, which has acquire semantics, and then loads the job pointer. A pop that succeeds read the pushed word, so it synchronizes with the push and reads its job pointer. The drain orders the writer's release against the passes: a writer that observed the popped bit, drained, and released knows that the pass of that pop ended before the release, because that pass began before the fence of the drain.

This argument is checked and not proved: Section 3.6 names the five memory models under which GenMC exhausts the executions of the drain and the resize clients, and the C11 model is among them RC11 [12].

2.5 The sleep and the futex contract

A worker that cannot grow the manifold, and a drain that waits on a held lane, back off on the lane word. The backoff keeps two locals, the nanosecond at its start and an exponent k, and no shared word holds a time. Inside the first unit of 2^u nanoseconds, with u as `EMBED_TURNSTILE_UNIT_SHIFT`, a step is a processor pause and no call. From one unit on, a step sets the sleeper bit with a fetch-or and calls the wait of the host on the lane word for at most 2^{u+k} nanoseconds, then k rises by one to at most `EMBED_TURNSTILE_K_MOST`, which is 20.

The host supplies three functions, and the headers declare them without a definition:

```
int embed_word_wait(embed_atomic_word *word, embed_atomic_word_value seen, embed_u64 nanoseconds);
void embed_word_wake(embed_atomic_word *word);
embed_u64 embed_clock_ns(void);
```

The contract of the wait is the futex contract futex [9]: the compare of the word against `seen` and the sleep are one step with respect to every wake on the word, and a wake ends each sleep on the word. The test host implements it on the Linux futex, on `WaitOnAddress` on Windows, on `__ulock_wait` on macOS, and as a timed sleep elsewhere. The futex compares 32 bits; on a little-endian host those are the low half of the lane word, which holds the three state bits and the low bits of the generation, and every leave, close and takeover changes them (Theorem 3.1, `changes_seen`). A build that defines none of the three functions fails at the link, and the error names them.

1.3 3. FORMAL VERIFICATION IN LEAN 4

The Lean project is `theory/embedded_types`. It builds with Lean `v4.35.0`, and the proofs workflow fails where any theorem closes with `sorry`. Bit-level theorems are stated on `BitVec` at 32 and at 64 bits and closed with `bv_decide`, which checks the statement on every value of the width through a SAT solver with a verified certificate checker. Transition-system theorems are closed by induction

over an inductive `Reach` relation. This section lists the definitions and the theorems as the files state them.

3.1 The lane word (`Manifold/LaneWord.lean`)

The operations of the C are definitions on `BitVec w`:

```
def busy (w : Nat) : BitVec w := 1
def closed (w : Nat) : BitVec w := 2
def sleeper (w : Nat) : BitVec w := 4
def unit (w : Nat) : BitVec w := 8
def generation {w : Nat} (x : BitVec w) : BitVec w := x >>> 3
def entered {w : Nat} (x : BitVec w) : BitVec w := x ||| busy w
def takes {w : Nat} (x : BitVec w) : Bool := x &&& busy w == 0
def left {w : Nat} (x : BitVec w) : BitVec w := x + (unit w - busy w)
def shut {w : Nat} (x : BitVec w) : BitVec w := x + (unit w + closed w)
def takenOver {w : Nat} (x : BitVec w) : BitVec w := x + (unit w - closed w)
def flagged {w : Nat} (x : BitVec w) : BitVec w := x ||| sleeper w
def unflagged {w : Nat} (x : BitVec w) : BitVec w := x &&& ~~~(sleeper w)
def low {w : Nat} (x : BitVec w) : BitVec w := x &&& 0xFFFFFFFF
```

Each theorem below exists at 64 bits and at 32 bits; the paper names the 64-bit form.

- `entry_takes_64`: $\forall x,\ x \wedge 1 = 0 \Rightarrow \mathrm{entered}(x) = x + \mathrm{busy} \ \wedge \ \mathrm{entered}(x) \wedge \neg\mathrm{busy} = x$. An entry on a free word sets exactly the busy bit and leaves every other field, the generation among them, as it was.
- `entry_refused_64`: $\forall x,\ x \wedge 1 = 1 \Rightarrow \mathrm{entered}(x) = x \wedge \neg\mathrm{takes}(x)$. An entry on a busy word changes nothing.
- `leave_64`: a leave on a word with busy set clears busy, keeps the closed and sleeper bits, and adds one to the generation.
- `close_64`: a close on a word with busy set and closed clear sets closed, keeps busy and the sleeper bit, and adds one to the generation.
- `takeover_64`: a takeover on a word with both bits set leaves the two low bits at busy alone, keeps the sleeper bit, and adds one to the generation.
- `changes_seen_64`: $\forall x,\ (x \wedge 1 = 1 \Rightarrow \mathrm{low}(\mathrm{left}(x)) \neq \mathrm{low}(x)) \wedge (x \wedge 3 = 1 \Rightarrow \mathrm{low}(\mathrm{shut}(x)) \neq \mathrm{low}(x)) \wedge (x \wedge 3 = 3 \Rightarrow \mathrm{low}(\mathrm{takenOver}(x)) \neq \mathrm{low}(x))$. Every operation that ends a hold, applied to a word in the state it applies to, changes the 32 bits that the futex compares.
- `sleeper_not_over_64` and `end_over_64`: the sleeper bit alone does not make a snapshot read as over, and a leave does.
- `pass_64`: $\forall x,\ x \wedge 1 = 0 \Rightarrow \mathrm{left}(\mathrm{entered}(x)) = x + \mathrm{unit} \wedge \mathrm{left}(\mathrm{entered}(x)) \wedge 1 = 0$. One pass adds exactly one unit to the word and returns the lane free.
- `roll_64`: $\forall x,\ x + \mathrm{unit} \cdot 2^{61} = x$. The generation rolls after 2^{w-3} counts.

- `classes_64` and `class_two_never_64`: the two low bits give the three lane states of `Open.lean` (free, held, closed), and no operation reaches the fourth class, closed with busy clear.

Theorem 1 below is the conjunction that the architecture needs from this file: **a lane word is exclusive**. Where a worker's fetch-or returns a value with busy clear, no other fetch-or returned busy clear between that worker's entry and its leave, because `entry_refused` holds for every intermediate value and `leave` is the only operation that clears busy on a held lane. This is a property of the atomic read-modify-write on one word, which the target guarantees (Axiom A1) and the Lean file states per operation.

3.2 The open count and the manifold (`Manifold/Open.lean`)

The manifold is a state with an open count `count`, a high-water mark `high`, and a lane function:

```
inductive Lane where
  | free | pass | held | sealing | closed
  | growing (fromClosed : Bool)

structure State where
  count : Nat
  high : Nat
  lane : Nat → Lane
```

`Step most` is an inductive relation with one constructor for each atomic operation of the C: the fetch-or of an entry, the fetch-add of a leave, the fetch-or of a contraction and its swap of `open` that lands or fails, the fetch-add that closes a sealing lane, the compare-and-swap of a growth from free or from closed, and the swap of `open` that lands or fails. Other steps can come between any two of them. `Reach most` is the reflexive-transitive closure from `rest`, the set of zeros. An invariant `Inv most s` is proved for every reachable state, and from it:

- `start_in_range`, `start_shift`: $\forall a\, n,\ 0 < n \Rightarrow (a \gg 6) \bmod n < n$, and the C expression `(address >> 6) % n` is that start.
- `spill_lane`, `spill_reaches`: the next lane of the C, o after the top and else the lane above, visits $(f + j) \bmod n$ at step j, so a spill from any start reaches each lane below n within n steps.
- `cascade_bounded`: $\forall c,\ 1 \le c \Rightarrow 1 \le \text{cascade}(c) \le c$. The cascade of contractions after a leave ends within $c - 1$ steps and never closes the last lane.
- `passes_below_open` (**Theorem 3.3**): for every reachable state s,

$$(\forall l,\ s.\text{lane}(l) = \text{pass} \Rightarrow l < s.\text{count}) \ \wedge\ (\forall l,\ s.\text{lane}(l) = \text{held} \Rightarrow l < s.\text{count}),$$

$$(\forall l,\ s.\text{lane}(l) = \text{closed} \Rightarrow s.\text{count} \le l) \ \wedge\ 1 \le s.\text{count} \le \text{most}.$$

Theorem 3.3 is what makes the snapshot of the drain complete: the drain reads `open` after its fence, and every lane that a pass can hold is below that value.

3.3 The queue word (`Manifold/Queue.lean`)

The queue word counts the workers between their addition and their subtraction. `queue_counts` proves $s.\text{word} = s.\text{waiting} \wedge 0 \leq s.\text{word}$ for every reachable state, and `empty_at_zero` proves that a word read at or below zero has no worker in the queue, which is the condition under which a leave contracts. `entry_balanced` proves that the entry of the C, with its local flag `queued`, adds at most once and subtracts only its own addition, so an entry that takes a lane leaves the queue word as it found it.

3.4 The drain and the sleeps (`Manifold/Drain.lean`, `Manifold/Wake.lean`)

The drain file models one lane with a holder, an ended set, and the busy, closed and generation fields. Its invariant is that a holder implies busy set and closed clear, and closed implies busy. The theorems:

- `held_until_end`: from a state where pass p holds the lane, each later state either has p in the lane with busy set, closed clear and the generation unchanged, or has p ended.
- `over_means_ended`: a drain that read p in the lane and later reads the lane over, where $\text{over}(\mathit{snapshot}, \mathit{now}) = \neg \mathit{now}.\text{busy} \vee \mathit{now}.\text{generation} \neq \mathit{snapshot}.\text{generation}$, saw p end.
- `drain_lane_complete`: with p the holder at the fence of the drain, a later read t, and a read u at which either t was free or closed or $\text{over}(t, u)$, the pass p has ended at u.

With Theorem 3.3, the loop of the drain over the open lanes ends with every pass of the fence ended. This is the correctness of the drain as a grace period in the sense of read-copy update RCU [10]: after the drain returns, no pass that began before the fence can still read the old pointer.

The wake file models the sleeper bit, a pending clear and an owed wake. Its theorem `no_lost_wake` states that in every reachable state a thread asleep on a value t has the word still at t with busy set, or a fetch-add owes it a clear and a wake, or the wake is out. A sleeper therefore sleeps only while the pass it waits for holds the lane, and the end of that pass wakes it. The theorem rests on the futex contract (Axiom A2): the compare and the sleep of the host are one step with respect to the wake.

3.5 The backoff (`Manifold/Backoff.lean`)

`shift_defined` proves that the shift of the C, one shifted left by the unit shift plus the exponent, is below 64 bits for every exponent the C reaches, so the C is defined. `limit_fits` and `ceilings` bound every wait at 2^{30} nanoseconds on a CPU host and 2^{20} on a device. `no_call_in_unit` and `wait_from_unit` prove that a step inside the first unit makes no call and a step from one unit on waits with the limit of k. `timespec_fits`, `microseconds_fit` and `milliseconds_fit` prove that each host conversion of the limit fits its argument type. `update_is_next` proves that the branchless increment `exponent += (exponent < EMBED_TURNSTILE_K_MOST)` is the function `nextK`.

3.6 The ticket ring (`TicketRing.lean`)

The stamp and the place are definitions on `BitVec` and on `Nat`:

```
def stamp (epoch ticket : BitVec 32) : BitVec 64 :=
  (epoch.zeroExtend 64 <<< 32) ||| ticket.zeroExtend 64
def ticketOf (s : BitVec 64) : BitVec 32 := (s &&& 0xffffffff#64).truncate 32
def epochOf (s : BitVec 64) : BitVec 32 := (s >>> 32).truncate 32
def place (shift ticket : Nat) : Nat := ticket % 2 ^ shift
```

Theorem 1 (non-aliasing). The theorems `stamp_ticket`, `stamp_epoch` and `stamp_injective` state

$$\forall e\, t,\ \mathrm{ticketOf}(\mathrm{stamp}(e,t)) = t\ \wedge\ \mathrm{epochOf}(\mathrm{stamp}(e,t)) = e,$$

$$\forall e_1\, t_1\, e_2\, t_2,\ \mathrm{stamp}(e_1,t_1) = \mathrm{stamp}(e_2,t_2) \Rightarrow e_1 = e_2 \wedge t_1 = t_2.$$

The theorems `place_lt`, `place_wrap`, `place_roll`, `place_mask` and `window_distinct` state

$$\forall s\, t,\ \mathrm{place}(s,t) < 2^s,$$

$$\forall s\, t,\ \mathrm{place}(s, t + 2^s) = \mathrm{place}(s,t),$$

$$\forall s\, t,\ s \le 32 \Rightarrow \mathrm{place}(s, t \bmod 2^{32}) = \mathrm{place}(s,t),$$

$$\forall s\, t,\ t \wedge (2^s - 1) = \mathrm{place}(s,t),$$

$$\forall s\, t_1\, t_2,\ t_1 < t_2 \wedge t_2 - t_1 < 2^s \Rightarrow \mathrm{place}(s,t_1) \neq \mathrm{place}(s,t_2).$$

The last of these is the non-aliasing theorem: two tickets fewer than 2^s apart stand at places of their own. The ring itself is a function from places to slots, and the pushes of a list are a fold:

```
def push (shift : Nat) (r : Ring) (ticket job : Nat) : Ring :=
  fun p => if p = place shift ticket then ⟨ticket, job⟩ else r p
def pop (shift : Nat) (r : Ring) (ticket : Nat) : Slot := r (place shift ticket)
def pushes (shift : Nat) (r : Ring) : List (Nat × Nat) → Ring
  | [] => r
  | (t, j) :: rest => pushes shift (push shift r t j) rest
```

`pop_push` proves $\mathrm{pop}(\mathrm{push}(r,t,j),t) = \langle t, j\rangle$, `pop_push_other` proves that a push at another place leaves a pop as it was, and `pop_pushes` proves

$$\forall l\, t\, j,\ (t,j) \in l\ \wedge\ \mathrm{Pairwise}_{\neq}(l) \Rightarrow \mathrm{pop}(\mathrm{pushes}(r,l),t) = \langle t, j\rangle,$$

where $\mathrm{Pairwise}_{\neq}(l)$ states that any two members of l have tickets at distinct places.

Theorem 2 (schedule confluence of a window). `pop_pushes` quantifies over every list l whose tickets stand at distinct places, and places no condition on the order of l. Any two interleavings of the pushes of one window are two such lists with the same members, and the theorem gives the same pop for every ticket of the window under both. The ring state after a window is therefore a function of the set of pushes and not of their schedule: every schedule projects to the same ring, which is the confluence the architecture claims. Two pushes to one place do not occur, because the grantor keeps the depth of the ring below 2^s (Axiom A3), and a pop never interleaves with the push of its own ticket, because the turnstile keeps each inside a pass and the drain of the producer orders the two (Section 2.4).

Theorem 3 (bounded rewind). A rewind of k tickets from a current ticket c is one pop at stamp$(e, c - k)$, and the artifact needs no second operation for it. For $k < 2^s$, `window_distinct` puts $c-k$ at a place of its own among the live tickets, `place_wrap` and `place_roll` make the place the same across the wrap of the ring and the roll of the 32-bit ticket, and `pop_pushes` gives the pop the job that the push of $c - k$ wrote. The pop is $O(1)$: one shift, one mask, two loads. The echoed stamp confirms the rewind landed, by `stamp_injective`: a slot that a ticket outside the window overwrote carries that ticket's stamp and not $c - k$. The retention window W is 2^s by `place_lt`, and an undo over the window is the same pop applied k times, each one an instance of `pop_pushes`.

Theorem 4 (single-event upset containment). `place_lt` holds for every natural number t, with no hypothesis on t. A stamp whose ticket field a single-event upset flipped in any bit still indexes a slot below 2^s, because the mask of the C is the modulus (`place_mask`). The slot access is therefore in bounds for every 64-bit value of the stamp, flipped or not. Detection is the read-back and the claim, Section 3.9: `flip_changes` proves that any nonzero mask changes a stamp, so a flip in the stamp field of a slot always fails the compare against the stamp the consumer holds, and `collision_detected` proves that a flip in the consumer's own ticket, which lands on the live slot of another ticket, collides with that ticket's owner at the claim and one of the two fails. `pop_stamp_free_of_job` proves that the stamp of a pop does not depend on the job; the job pointer is covered by its own parity bit in the stamp, Section 3.10, so a single-bit flip there fails the read and no caller runs the job.

3.7 Model checks (`test/prove`)

Three GenMC clients run under the sequentially consistent, TSO, RC11, IMM and ARMv8 models. The ticket ring client grants and pushes one slot, then runs two workers that pop the same stamp inside passes, as the owner and a worker with a flipped ticket collide, while a writer drains and releases. It asserts that exactly one pop succeeds and reads the job the push stored, that the failed pop finds the popped bit, that the release succeeds only after a pop did, and that the final word carries the flags of the sequence, the lane index of the pop, and the fields of the stamp. The drain client runs two workers through a set of one lane on one start lane while a writer swaps a published word and drains; the second worker finds the lane busy and grows the set. GenMC exhausts every execution the model allows and reports a failed assertion, a data race or a liveness violation. The assertions are that a worker that reads the old word reads it before the writer releases it, that no thread stays asleep with no wake to come, and that each lane is free or closed at the end. The resize client runs two workers through two passes each while a writer drains, so

each growth races each contraction, and asserts that each pass holds a lane below the open count, that the drain ends, and that at the end no lane below the open count is closed or busy. A client that passes under one model and fails under another names the order that the code needs; all three clients pass under all five.

3.8 Coarse-grained refinement

Every theorem of Sections 3.1 to 3.6 is stated over steps that are atomic operations of the C: a fetch-or, a fetch-add, a compare-and-swap, a load after a fence. The consumer of the turnstile does not reason at that grain. It reasons at the grain of a *pass*: an entry, some work, a leave. Theorems 3.3 and `drain_lane_complete` together say that a drain ends with every pass of its fence ended, and that is the one fact a consumer needs: the step "enter, work, leave" is indivisible with respect to the drain. A consumer therefore takes the pass as its smallest transition and proves its own invariants by induction over sequences of passes and drains, and it inherits the turnstile's proof by importing it. Section 4 shows a consumer that does exactly this.

3.9 The claim of a slot (`Claim.lean`)

A worker acts on a slot only after one read-modify-write on it succeeds. The push is a compare-and-swap from empty to the stamp; the claim is a compare-and-swap from the stamp to the stamp with a consumed mark and the id of the pass. Each succeeds for one caller and fails for every other, which is the contract `cas_contract_64` proves for the arms. The model is a slot with three states and two operations:

```
inductive Slot where
  | empty
  | pushed (stamp : Nat)
  | consumed (stamp : Nat) (pass : Nat)

def push (slot : Slot) (stamp : Nat) : Slot × Bool :=
  match slot with
  | .empty => (.pushed stamp, true)
  | _ => (slot, false)

def claim (slot : Slot) (stamp pass : Nat) : Slot × Bool :=
  match slot with
  | .pushed t => if t = stamp then (.consumed stamp pass, true) else (slot, false)
  | _ => (slot, false)
```

The theorems:

- `flip_changes` and `bit_flip_changes`: $\forall x\, m,\ m \neq 0 \Rightarrow x \oplus m \neq x$, and $\forall x\, i,\ i < 64 \Rightarrow x \oplus 2^i \neq x$, by `bv_decide`. Any flip in a stamp field fails the stamp compare.
- `pop_stamp_free_of_job`: the stamp that a pop returns is the same after any change to any job in the ring.

- `push_once`, `claim_pushed`, `claim_other_fails`, `claim_once`: a push lands on an empty slot alone; a claim with the stamp of a pushed slot lands; a claim with another stamp fails; after one claim lands, no claim lands.
- `claims_at_most_one`: $\forall\, slot\, l,\ \text{count}(\text{claims}(slot, l)) \leq 1$. Any sequence of claims on one slot lands at most once, so no job runs twice.
- `collision_detected`: with $c_1 = \text{claim}(\text{pushed}(s), s, p)$ and $c_2 = \text{claim}(c_1.\text{slot}, s, q)$, $\forall\, s\, p\, q,\ c_1.\text{landed} \wedge \neg\, c_2.\text{landed}$. The owner of a ticket and a worker whose flipped ticket landed on the same slot collide; the second claim fails and raises the fault.
- `claim_in_pass` and `claim_drained`: a claim step is a step of a combined state, one lane of `Drain.lean` and one slot, and the step exists only where the lane's holder is the claiming pass. So a consumed slot names a pass that held the lane at the claim, and `over_means_ended` applies to that pass: a drain that read the lane held then waits until the claimant's pass ends.
- `drain_waits_while_held`: $\forall\, s\, u\, p,\ \text{Inv}(s) \wedge s.\text{holder} = p \wedge \text{Follows}(s, u) \wedge \neg u.\text{ended}(p) \Rightarrow \neg\, \text{over}(s, u)$. The drain never reads a lane over while the pass it read there has not ended. This is the liveness condition of the drain stated as a theorem: the drain terminates exactly when every pass of its fence terminates.

The claim binds the identity to the gate. The lane that a worker holds is the one its constant address selected, `entry_takes` gives that lane one holder, and the claim step requires that holder. A flipped worker therefore claims from inside its own pass, on its own lane, and the collision with the owner is a collision of two claims on one slot, which `claim_once` resolves to one success. The C runs this protocol: every transition of `embed_ticket_ring.h` is the compare-and-swap of Section 2.2, the pop writes the lane index into the word, and the abstract `claim` of this file is the pop transition with the lane as the pass id.

3.10 The flag matrix (`Flags.lean`)

The flag field is the slot state machine, and `Flags.lean` is its matrix, every cell a `bv_decide` theorem over the 64-bit word:

- `set_keeps_fields`, `set_keeps_others`: a transition preserves the ticket, the epoch, and every other flag bit. $\forall\, x\, f,\ f \wedge \text{flagMask} = f \Rightarrow \text{fields}(x \vee f) = \text{fields}(x)$.
- `set_once`: $\forall\, x\, f,\ f \neq 0 \wedge x \wedge f = 0 \Rightarrow x \vee f \neq x$. The word after a transition differs from the word before, so the compare-and-swap that expects the word before succeeds exactly once, and a repeat fails.
- `states_distinct`, `states_keep_fields`: the five state words granted, pushed, popped, replayed, released are pairwise distinct along the sequence and each preserves the fields of the stamp.
- `independent`: two flag bits set in either order give one word, and each stays set. Sixteen flags are sixteen independent one-success gates on one word.

- `lane_recorded`: $\forall s\, l,\ l < 128 \Rightarrow (\text{statePopped}(s, l) \wedge \text{laneMask}) \gg 40 = l$. The lane index of the pop is recoverable from the popped word.
- `corruption_refuses`: $\forall x\, m,\ m \neq 0 \Rightarrow x \oplus m \neq x$. Any bit flip in a slot word fails the next transition.
- `states_reserved_clear`, `invalid_input_refuses`, `valid_input_expects`: no state word has the reserved bit; an input stamp with a flag bit set expects a value with the reserved bit, so its transition fails on every state word; an input stamp with a clear flag field expects exactly the stamp with the prior flags. So a flip in any of the 64 bits of a held stamp fails the transition: a flip in the ticket or the epoch expects another stamp's word, a flip in the flag field expects the reserved bit.
- `guarded_refuses`: a transition that reads the current value first and fails its state test expects the current value with the reserved bit inverted, which differs from every current value, the zero word of an empty slot among them. The pop, the replay and the release are of this form, because the lane and parity bits of the current value are not known to the caller in advance.
- `job_flip_inverts_parity`: $\forall j i,\ i < 64 \Rightarrow \text{parity}(j \oplus 2^i) \neq \text{parity}(j)$, where parity is the six-step fold of the C. `parity_recorded`: the pushed and the popped words carry the parity the push stored, where the pop reads it. A single-bit error in the job pointer therefore fails the check at the pop or the replay, and the pop returns no job.

Theorem 4 is therefore complete for the whole slot: a flip in the slot word fails the next transition on it (`corruption_refuses`), a flip in a held stamp fails every transition from it (`invalid_input_refuses` with `fields_tell_tickets`), and a flip in the job pointer fails the parity check at the read (`job_flip_inverts_parity`). The suite exercises the last two directly: it flips each of the 64 bits of a held stamp in turn and asserts that the pop fails on every one, and it flips each of the 64 bits of a stored job pointer in turn and asserts that the pop and the replay refuse the job and mark the reserved bit.

3.11 The device arm (`Device.lean`)

Under nvcc one definition of every atomic, turnstile and ring function compiles into the host pass and the device pass, and the device pass takes arm 1 of the atomics: `atomicOr`, `atomicAnd`, `atomicAdd`, `atomicExch`, `atomicCAS` and `__threadfence`. `Device.lean` models each primitive as one read-modify-write that returns the old value, and proves that the device arm instantiates every model above with its own constants:

- `or_is_entered`, `or_is_flagged`, `and_is_unflagged`, `add_is_left`, `add_is_shut`: the device fetch-or, fetch-and and fetch-add on a lane word are exactly `entered`, `flagged`, `unflagged`, `left` and `shut` of `LaneWord.lean`, each returning the old value. Every lane word theorem of Section 3.1 therefore holds on the device.
- `cas_is_cas`: the device compare-and-swap is the `cas` of `Atomic.lean`, so `cas_contract` holds on the device, and with it every one-success theorem of the claim and the flag matrix. `cas_is_transition` states a flag transition on the device as one `atomicCAS` from the

expected value to the new value. `raise_is_cas_loop` states one round of the raise loop on the device as `raiseRound`.

- `lane_is_256`, `lane_u32s_is_8`: the lane on a device is 256 bits, eight 32-bit registers, on every word width. `unit_shift_valid`, `ceiling_is_2_20`: the backoff unit shift on a device is 0, within the bound of `shift_defined`, and the ceiling of a wait is 2^{20} ns.

The transition systems of Sections 3.2 to 3.4 and 3.9 take each operation as one step and never name an arm, so they hold on the device by the instantiation above. What remains for the device is Axiom A1 in its device form: that `__threadfence` orders the writes before it ahead of the reads after it at device scope, as the sequentially consistent fence does on the host. That is the same axiom under which the host arms are proved, and not a weaker one. The device suite exercises the lane's device arm where a CUDA part is present.

`LaneWord.lean` also gains `fewer_ends_move_64` and `fewer_ends_move_32`: $\forall x\, n,\ n \neq 0 \wedge n < 2^{w-3} \Rightarrow \mathrm{generation}(x + n \cdot \mathrm{unit}) \neq \mathrm{generation}(x)$. Fewer passes than the roll always move the generation, so the drain's misread of Section 6 needs exactly the roll count of passes between its two reads of one lane, and not one fewer.

1.4 4. CASE STUDY: THE CELLBLOCK MEMORY MANAGER OVER THE TURNSTILE

The verified consumer is `locus_carcerum`, the cellblock of the MMgr memory manager. A cellblock is a caller's storage divided into two tiers that grow toward each other, with the two boundaries as the two halves of one atomic word. A take moves its own half by one fetch-add inside a pass through the cellblock's turnstiles; a writer, which compacts, grows or moves the cellblock, works alone. The C is one `EmbedTurnstiles` as the last member of the cellblock state, zero at rest:

```
typedef struct
{
    uint8_t *const base;
    const size_t size;
    embed_atomic_usize bounds;
    ...
    embed_atomic_word closed;
    embed_atomic_usize sink;
    EmbedTurnstiles turnstiles;
} SeriesCellularum;
```

A pass (`carcer_attempted`) enters the turnstiles at the address of the caller's own word, reads the closed bit, and works on the cellblock where the bit is clear or on a sink word where it is set, then leaves:

```
const unsigned int at = embed_turnstile_enter(&series->turnstiles,
                                              (embed_atomic_uptr_value)(uintptr_t)missed);
const size_t closed = carcer_all((embed_atomic_word_load(&series->closed) & CARCER_CLOSED) != 0u);
...
const uintptr_t answer = work(series, context, permit);
embed_turnstile_leave(&series->turnstiles, at);
```

A writer (`carcer_shut`) takes the closed bit with a fetch-or, backs off on it with `embed_turnstile_backed_off` while another writer holds it, issues the fence, and drains:

```
EMBED_ATOMIC_ORDERED();
embed_turnstiles_drain(&series->turnstiles);
carcer_settled(series);
```

The proof of the cellblock is the Lean project `theory/locus_carcerum` of MMgr, and its lakefile requires `EmbeddedTypes`, the project of Section 3. Four of its files are the refinement of Section 3.8 made concrete.

- `Order.lean` imports `EmbeddedTypes.Manifold.Open` and `EmbeddedTypes.Manifold.Drain`. It carries `passes_below_open` as `pass_lane_below_open` and `drain_lane_complete` as `drain_ends_pass`, and states the C11 fence rule for the two sequentially consistent fences of the pass and the drain as the two hypotheses of an execution. Its theorem `drain_or_closed` is the disjunction the consumer needs: the pass reads the closed bit set, or the drain reads the pass's lane busy and waits it out. Its theorem `unfenced_admits` exhibits the store-buffered execution that violates the disjunction with the fences removed, so the fences are necessary and not decorative.
- `Pass.lean` imports `EmbeddedTypes.Manifold.Drain`. Its `drained` step takes every pass idle as its hypothesis, and `drained_lane` discharges that hypothesis from `drain_lane_complete`. Its theorem `alone_exclusive` states that while a writer works alone no pass works on the cellblock, by induction over `Reach` with the pass as one whole step.
- `Sleep.lean` imports `EmbeddedTypes.Manifold.Backoff`. `wait_bounded` carries `limit_fits`, and `gated_woken` proves that a sleeper on the closed word, gated on the closed bit as the C gates it, is never asleep with the writer at rest. `ungated_sleeps` exhibits the lost wake that the gate prevents.
- `Futex.lean` proves the host contract that Section 2.5 trusts, against the model wait that GenMC runs: `real_in_model` and `model_in_real` state that the real wait and the model reach the same states, `whole_in_low` that a model on the 32-bit compare holds every host, and `low_sleeps` that a model on the whole word misses a sleep that Linux takes.

The cellblock is a memory manager and not a kernel, but it is the composition the architecture promises: page-sized storage with a persistent and a temporary tier, a scheduler of passes over it, and a writer that retires storage behind a drain, all proved by sequential induction over passes with the turnstile theorems imported and not restated. A fault handler or an inter-process channel over the ring composes the same way: the events are the list of pushes of Section 3.6, the retirement of storage is the drain, and the sleep of a waiter is the futex contract that `Futex.lean` has proved sound against the model.

1.5 5. RELATED WORK

Vyukov's bounded MPMC queue Vyukov [4] stores a sequence number in each cell. A producer loads the enqueue position, loads the sequence of the cell, and claims the cell with a compare-and-swap on the position when the sequence matches; it then stores the payload and publishes by a release store of the sequence. The queue needs no external guard and admits concurrent producers

and consumers on distinct cells. The cost is one compare-and-swap loop per operation and one atomic per cell, and the proof of the queue is the proof of the cell protocol. The ticket ring has no per-slot atomic and no loop; it moves the loop into the entry of the turnstile, where a spill over the lanes replaces a retry over the position, and it moves the per-slot sequence into the stamp that the push records. The ring does not admit two producers to one slot at all; it requires the grantor to keep the window.

The LMAX Disruptor Disruptor [5] is a ring with a monotonic sequence, a sequence barrier and a wait strategy, and its slots are pre-allocated objects. Its claim is a compare-and-swap or a single-writer increment on the cursor, and its consumers track the cursor through the barrier. The turnstile has no cursor; the only shared counters are the open count and the queue word of the manifold, and a reader never reads a producer's position. The Disruptor's wait strategies are a configuration surface; the turnstile has one backoff, bounded by the theorems of Section 3.5, and one host contract.

The Linux printk ring buffer printk [6] is a lockless ring with a descriptor ring and a data ring, with state transitions on descriptors by compare-and-swap, and it tolerates a writer that dies mid-record. The ticket ring does not tolerate a dead holder: a pass that never leaves holds its lane and the drain waits without end (Section 6). The printk design pays for that tolerance in the descriptor protocol; the ticket ring pays for its simplicity in the trust it places in the caller.

io_uring io_uring [7] is a pair of single-producer single-consumer rings between a process and the kernel, with head and tail indices and a mask over a power of two. The mask over a power of two is the same device as `place_mask`. io_uring has one producer and one consumer per ring by construction; the ticket ring admits many of each, and the turnstile is what pays for that.

seL4 seL4 [8] is a verified kernel with a proof of functional correctness from the specification to the binary. The artifact here is not a kernel and makes no such claim. Its proofs are of a data structure and a synchronization primitive, and Section 4 shows a memory manager refined onto them with the turnstile theorems imported whole. **MCS locks** MCS [11] give each waiter its own cache line and hand the lock down a queue; the turnstile gives each lane its own cache line, but a lane is not a lock: many passes proceed at once on distinct lanes and the only thread that waits for a pass is a drain. In that respect the turnstile is closer to read-copy update RCU [10], with the drain as the grace period and the lane word as the per-reader state that the grace period waits on.

1.6 6. LIMITATIONS AND CONCLUSION

The trust boundary of the architecture is three axioms, one per source, and the paper has named each where it is used.

- **Axiom A1, the atomics of the target.** Each `embed_atomic_*` operation is one indivisible step with the memory order of its arm, and the sequentially consistent fence orders as C11 states. A target with none of the four arms fails the build at `#error`. The Lean model takes each operation as one step and does not model the compiler or the hardware.
- **Axiom A2, the wait of the host.** `embed_word_wait` compares and sleeps as one step with respect to `embed_word_wake`, and a wake ends every sleep on the word. The Lean theorem `no_lost_wake` and the GenMC liveness check both assume this contract. A host whose wait is a timed sleep, the fallback arm, keeps safety and loses the bound on the sleep.

- **Axiom A3, the depth bound.** The grantor of tickets keeps the number of tickets between the oldest unpopped and the newest granted below 2^s, and grants each ticket once. `window_distinct` and `pop_pushes` are conditional on it. The ring detects a violation only through the stamp (below), and the turnstile does not detect it at all.

Three facts that read like limits are theorems, and the paper states them as such rather than as limits.

- **Which worker runs a job.** The host orders the queue, and the stamp carries that order: epoch and ticket are the causal and temporal link of a job, and the slot records them on every transition. A worker is therefore interchangeable with any other. Where a worker's held ticket is corrupted onto another live ticket, it pops that ticket's job and the owner of that ticket finds the popped bit; the job ran once, under the stamp that the host assigned it, and the only consequence is which fungible worker ran it. `collision_detected` and `claims_at_most_one` are the whole of what the architecture has to say, and they are enough.
- **A pass that does not leave.** `drain_waits_while_held` states the liveness condition of the drain exactly: it returns when every pass of its fence has ended, and not before. That is the contract a reader signs at the gate.
- **The generation.** `roll_64` gives the period of the generation and `fewer_ends_move_64` proves that any count of passes short of it moves the generation. A drain's read of a lane is therefore exact at every count below 2^{w-3} passes between two reads, which is the bound, stated.

The one limit of the artifact is inter-process use. The Lean model of the wait does not distinguish a shared mapping from one process. The Linux futex gives the contract across processes for a shared mapping, and the lane words and the slots must be in that mapping; that is the host's property, Axiom A2, and not a theorem here.

In sum: the turnstile and the ticket ring separate the guard from the storage so that each can be proved on its own. The lane word, the open count, the queue word, the backoff, the drain and the sleeps are theorems of Lean 4 over a transition system whose steps are the atomic operations of the C. The stamp, the place, the window, the pushes and the claim of the ring are theorems over a function from places to slots, and the window theorem is what makes the ring a causal log, a rewind one pop, and an upset one compare or one failed claim. The flag matrix makes every transition of a slot one compare-and-swap that succeeds once and fails on any corruption, in the word or in the hand. The two proofs meet at one obligation, that a pass is exclusive with respect to the drain, GenMC checks that meeting point under five memory models, and a verified memory manager takes it as one whole step. Everything beyond that is an axiom, named.

BIBLIOGRAPHY

[1] ISO/IEC. *ISO/IEC 9899:2011, Programming languages, C*. International Organization for Standardization, Geneva, 2011.

[2] L. de Moura, S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE 28, LNCS 12699, 625–635, 2021, doi:10.1007/978-3-030-79876-5_37.

[3] M. Kokologiannakis, V. Vafeiadis. *GenMC: A Model Checker for Weak Memory Models*. CAV 2021, LNCS 12759, 427–440, 2021, doi:10.1007/978-3-030-81685-8_20.

[4] D. Vyukov. *Bounded MPMC queue*. 1024cores.net, 2011. https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue.

[5] M. Thompson, D. Farley, M. Barker, P. Gee, A. Stewart. *Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads*. LMAX technical paper, 2011.

[6] J. Ogness. *printk_ringbuffer: a lockless ringbuffer for printk*. Linux kernel, kernel/printk/printk_ringbuffer.c and its header comment, 2020.

[7] J. Axboe. *Efficient IO with io_uring*. 2019. https://kernel.dk/io_uring.pdf.

[8] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, S. Winwood. *seL4: Formal Verification of an OS Kernel*. SOSP 2009, 207–220, doi:10.1145/1629575.1629596.

[9] U. Drepper. *Futexes Are Tricky*. Red Hat, 2011. Also `futex(2)`, Linux Programmer's Manual.

[10] P. E. McKenney, J. D. Slingwine. *Read-Copy Update: Using Execution History to Solve Concurrency Problems*. Parallel and Distributed Computing and Systems, 509–518, 1998.

[11] J. M. Mellor-Crummey, M. L. Scott. *Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors*. ACM TOCS 9(1), 21–65, 1991, doi:10.1145/103727.103729.

[12] O. Lahav, V. Vafeiadis, J. Kang, C.-K. Hur, D. Dreyer. *Repairing Sequential Consistency in C/C++11*. PLDI 2017, 618–632, doi:10.1145/3062341.3062352.